

Relatório de Evidências — Etapa 3

Sistema: Controle de Atendimentos (Painel de Demandas e Ações orientado a eventos)

Domínio: Atendimento

Integrante: Marcos Moura Guedes · **Matrícula:** 2019200067

Sistema evoluído para arquitetura orientada a eventos e especializado para o domínio de atendimento. As capturas abaixo comprovam o funcionamento completo do fluxo Evento → API → Demanda, além da documentação e do versionamento.

Evidências

1 frontend personalizado

Controle de Atendimentos

Sistema de atendimento orientado a eventos: cadastro manual, eventos externos e geração automática de demandas.

Cadastrar demanda

Título

Categoria

Descrição

Status

Responsável

Data

Salvar demanda

Resumo

Total: 5
Pendentes: 3
Em andamento: 1
Concluídas: 1

Demandas cadastradas

Carregar dados

Foram carregadas 5 demandas.

- **Responder solicitacao de cliente**
Categoria: Solicitação
Descrição: Cliente pediu segunda via de boleto pelo formulario web
Status: pendente
Responsável: Marina
Data: 2026-04-11
[Excluir](#)
- **Tratar reclamacao de atendimento**
Categoria: Reclamação
Descrição: Cliente relatou demora no retorno do suporte
Status: em andamento
Responsável: Carlos
Data: 2026-04-11
[Excluir](#)
- **Esclarecer duvida sobre servico**
Categoria: Suporte
Descrição: Cliente perguntou sobre horarios de funcionamento
Status: concluida
Responsável: Luciana
Data: 2026-04-11
[Excluir](#)
- **Atendimento gerado por evento: solicitacao**
Categoria: Solicitação
Descrição: Evento recebido de formulario-web: segunda via de boleto
Status: pendente
Responsável: formulario-web
Data: 2026-06-01
[Excluir](#)
- **Atendimento gerado por evento: solicitacao**
Categoria: Solicitação
Descrição: Evento recebido de formulario-web: segunda via de boleto
Status: pendente
Responsável: formulario-web
Data: 2026-06-01
[Excluir](#)

Eventos Recebidos

Foram carregados 2 eventos.

- **formulario-web**
Tipo: solicitacao
Valor: segunda via de boleto
Data: 2026-06-01
- **formulario-web**
Tipo: solicitacao
Valor: segunda via de boleto
Data: 2026-06-01

Resumo

Total: 5

Pendentes: 3

Em andamento: 1

Concluídas: 1

Demandas cadastradas

Carregar dados

Foram carregadas 5 demandas.

- **Responder solicitacao de cliente**

Categoria: Solicitação

Descrição: Cliente pediu segunda via de boleto pelo formulario web

Status: pendente

Responsável: Marina

Data: 2026-04-11

Excluir

- **Tratar reclamacao de atendimento**

Categoria: Reclamação

Descrição: Cliente relatou demora no retorno do suporte

Status: em andamento

Responsável: Carlos

Data: 2026-04-11

Excluir

- **Esclarecer duvida sobre servico**

Categoria: Suporte

Descrição: Cliente perguntou sobre horarios de funcionamento

Status: concluída

Responsável: Luciana

Data: 2026-04-11

Excluir

- **Atendimento gerado por evento: solicitacao**

Categoria: Solicitação

Descrição: Evento recebido de formulario-web: segunda via de boleto

Status: pendente

Responsável: formulario-web

Data: 2026-06-01

Excluir

- **Atendimento gerado por evento: solicitacao**

Categoria: Solicitação

Descrição: Evento recebido de formulario-web: segunda via de boleto

Status: pendente

Responsável: formulario-web

Data: 2026-06-01

Excluir

Eventos Recebidos

Foram carregados 2 eventos.

- **formulario-web**
Tipo: solicitacao
Valor: segunda via de boleto
Data: 2026-06-01
- **formulario-web**
Tipo: solicitacao
Valor: segunda via de boleto
Data: 2026-06-01

5 rota POST /event

Controle de Atendimentos API 3.0.0 OAS 3.1

/openapi.json

API de atendimento orientada a eventos (Etapa 3)

default

GET	/health	Health	▼
GET	/demands	Get Demands	▼
POST	/demands	Create Demand Route	▼
GET	/summary	Get Summary Route	▼
PUT	/demands/{demand_id}	Update Demand Route	▼
DELETE	/demands/{demand_id}	Delete Demand Route	▼
GET	/events	Get Events	▼

POST /event: Receive Event

Parameters Cancel Reset

No parameters

Request body required application/json ▼

Edit Value Schema

```
{
  "source": "formulario-web",
  "type": "solicitacao",
  "value": "segunda via de boleto",
  "created_at": "2026-06-01"
}
```

Execute

Responses

Code	Description	Links
200	Successful Response	No links
	Media type application/json ▼ Controls Accept header.	
	Example Value Schema	
	<pre>"string"</pre>	
422	Validation Error	No links
	Media type application/json ▼	
	Example Value Schema	
	<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	

Schemas

DemandCreate > Expand all object

DemandUpdate > Expand all object

EventCreate > Expand all object

HTTPValidationError > Expand all object

ValidationError > Expand all object

7 demanda gerada automaticamente

Atendimento gerado por evento: solicitacao

Categoria: Solicitação

Descrição: Evento recebido de formulario-web: segunda via de boleto

Status: pendente

Responsável: formulario-web

Data: 2026-06-01

Excluir

Mini AI-Native Dev Lab

Objetivo

Laboratório local full stack com frontend, backend, API, banco e uso inicial de agente de IA no editor.

Como executar

docker compose up --build

Endereços

Frontend: <http://localhost:8080> API: <http://localhost:8000/projects> Docs da API: <http://localhost:8000/docs>

Controle de Atendimentos

Objetivo

Sistema da disciplina Projeto de Extensão em Software Full Stack, especializado para o domínio de **Atendimento** (Etapa 3). Além do cadastro manual de demandas, o sistema é orientado a eventos: recebe eventos externos e os transforma automaticamente em demandas de atendimento.

Domínio escolhido

Atendimento ao cliente. As demandas representam atendimentos e são organizadas nas categorias **Suporte**, **Reclamação** e **Solicitação**.

Funcionalidades

- cadastrar demandas manualmente
- listar demandas
- excluir demandas
- exibir resumo por status
- receber eventos externos (`POST /event`)
- transformar eventos em demandas automaticamente
- registrar e listar eventos recebidos (`GET /events`)
- exibir eventos e demandas no frontend

Fluxo de eventos

```
Evento → POST /event → registra na tabela events
                        → convert_event_to_demand() gera a demanda
                        → salva a demanda → retorna sucesso
Frontend → "Carregar dados" → exibe demandas, resumo e eventos
```

Exemplo de evento (domínio Atendimento)

```
{
  "source": "formulario-web",
  "type": "solicitacao",
  "value": "segunda via de boleto",
  "created_at": "2026-06-01"
}
```

O tipo do evento é mapeado para a categoria da demanda (`solicitacao` → Solicitação, `reclamacao` → Reclamação, `duvida` → Suporte).

Tecnologias

- FastAPI
- SQLite
- HTML
- CSS
- JavaScript
- Docker
- Docker Compose
- Git

Endpoints principais

- `GET /health` — status da API
- `GET /demands` — lista demandas
- `POST /demands` — cria demanda manual
- `PUT /demands/{id}` — atualiza demanda
- `DELETE /demands/{id}` — remove demanda
- `GET /summary` — resumo por status
- `POST /event` — recebe evento e gera demanda
- `GET /events` — lista eventos recebidos

Como executar

docker compose up --build

Arquitetura do Controle de Atendimentos

Este projeto possui uma arquitetura simples de aplicação web full stack, especializada para o domínio de Atendimento e orientada a eventos (Etapa 3).

Domínio

Atendimento ao cliente. As demandas representam atendimentos, classificados em Suporte, Reclamação e Solicitação. Eventos externos (formulário web, chat, etc.) geram demandas automaticamente.

Frontend

O frontend foi construído com HTML, CSS e JavaScript puro. Ele exibe o formulário de cadastro, a lista de demandas, o painel resumo e a seção de eventos recebidos. As funções `fetchEvents()` e `renderEvents()` carregam e exibem os eventos junto com as demandas e o resumo.

Backend

O backend foi construído com FastAPI. Ele oferece rotas para listar, criar, atualizar, excluir demandas e gerar resumo, além das rotas orientadas a eventos: - `POST /event` — registra o evento e gera uma demanda automaticamente. - `GET /events` — lista os eventos recebidos.

A função `convert_event_to_demand()` em `services.py` transforma um evento em uma demanda, mapeando o tipo do evento para a categoria do domínio.

Fluxo de eventos

```
Evento → API (POST /event) → Backend (registra evento + converte em demanda)
      → Banco (events + demands) → Frontend (exibe eventos e demandas)
```

Banco de dados

O banco utilizado é SQLite. Ele armazena duas tabelas: - `demands` — demandas cadastradas (manuais ou geradas por eventos). - `events` — eventos externos recebidos (source, type, value, created_at).

Infraestrutura

O sistema roda localmente com Docker e Docker Compose. Isso permite que o ambiente funcione de forma reproduzível.

Papel da IA

A IA pode ser usada como apoio para explicar, revisar e comparar soluções. Nesta etapa, a referência principal é o gabarito oficial do projeto.

10 historico Git

```
$ git log --oneline
0dc1410 especializa sistema para atendimento
21d3c06 integra eventos ao frontend
fc0353d implementa rota post event
6f98823 adiciona tabela de eventos
3fd857e Atualiza a referência do script app.js para incluir um parâmetro de versão, garantindo que a versão mais recente do arquivo seja carregada no frontend.
9278312 Implementa a API do Painel de Demandas e Ações com rotas para gerenciar demandas, incluindo criação, atualização, exclusão e resumo. Adiciona documentação de arquitetura e atualiza o README com informações sobre o sistema. O frontend é ajustado para interagir com a nova API.
66257a0 Atualiza e finaliza a documentação de arquitetura do projeto, descrevendo a estrutura do frontend, backend, banco de dados e Docker.
be7b5b3 Primeiro passo
```

11 terminal executando: docker compose up --build

```
$ docker compose up --build
Image mini-lab-backend Building
#1 [internal] load local bake definitions
#1 reading from stdin 549B 0.0s done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 252B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.12-slim
#3 DONE 0.8s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 465B done
#5 DONE 0.0s

#6 [1/5] FROM docker.io/library/python:3.12-slim@sha256:d764629ce0ddd8c71fd371e9901efb324a95789d2315a47db7e4d27e78f1b0e9
#6 resolve docker.io/library/python:3.12-slim@sha256:d764629ce0ddd8c71fd371e9901efb324a95789d2315a47db7e4d27e78f1b0e9
0.0s done
#6 DONE 0.0s

#7 [4/5] RUN pip install --no-cache-dir -r requirements.txt
#7 CACHED

#8 [2/5] WORKDIR /app
#8 CACHED

#9 [3/5] COPY requirements.txt .
#9 CACHED

#10 [5/5] COPY app ./app
#10 CACHED

#11 exporting to image
#11 exporting layers done
#11 exporting manifest sha256:83a3656fbcfb9eb8dbca9eef228cec643b0c3ff14e6398737f44608e3b83cc4 done
#11 exporting config sha256:bed3513669680f7d9fcd01af83ade27e3e593e01117e54023ca5cc8386e72fda done
#11 exporting attestation manifest sha256:cfcef92fe452ca3e83b3ccce56e67e28ee9daabce6dac67eedef3fb4722e6a4 0.0s done
#11 exporting manifest list sha256:ca1f7c8d1316673a33f15e37cc1046f5e276ae4ef10acebec02e91047202892f
#11 exporting manifest list sha256:ca1f7c8d1316673a33f15e37cc1046f5e276ae4ef10acebec02e91047202892f 0.0s done
#11 naming to docker.io/library/mini-lab-backend:latest done
#11 unpacking to docker.io/library/mini-lab-backend:latest 0.0s done
#11 DONE 0.1s

#12 resolving provenance for metadata file
#12 DONE 0.0s
Image mini-lab-backend Built
Container mini_ai_lab_backend Recreate
Container mini_ai_lab_backend Recreated
Container mini_ai_lab_frontend Running
Attaching to mini_ai_lab_backend, mini_ai_lab_frontend
Container mini_ai_lab_backend Starting
Container mini_ai_lab_backend Started
mini_ai_lab_backend | INFO: Started server process [1]
mini_ai_lab_backend | INFO: Waiting for application startup.
mini_ai_lab_backend | INFO: Application startup complete.
mini_ai_lab_backend | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
mini_ai_lab_backend | INFO: 172.18.0.1:45492 - "GET /health HTTP/1.1" 200 OK
mini_ai_lab_backend | INFO: 172.18.0.1:44522 - "GET /health HTTP/1.1" 200 OK
mini_ai_lab_backend | INFO: 172.18.0.1:44526 - "GET /demands HTTP/1.1" 200 OK
mini_ai_lab_backend | INFO: 172.18.0.1:44532 - "GET /events HTTP/1.1" 200 OK
mini_ai_lab_backend | INFO: 172.18.0.1:44536 - "GET /docs HTTP/1.1" 200 OK
mini_ai_lab_frontend | 172.18.0.1 - - [14/Jun/2026:15:01:24 +0000] "GET / HTTP/1.1" 200 2238 "-" "curl/8.18.0" "-"
mini_ai_lab_backend | INFO: 172.18.0.1:51900 - "GET /health HTTP/1.1" 200 OK
```